Correct answer: **B. Overwrites history** ✅

Explanation:
git push --force (force push) **rewrites the commit history of the remote repository**.

- Normally, Git **prevents pushing if the remote branch has new commits** you don't have.
- When you use **force push**, Git **ignores this safety check** and **replaces the remote history with your local history**.

⚠️ **Why this is dangerous:**

- It can **delete other developers' commits** on the remote branch.
- Team members may lose work or face merge conflicts.

Example:
If the remote branch has:

A → B → C

and your local branch has:

A → B → D

Using **force push** will change the remote to:

A → B → D

Commit **C disappears from the remote history**.

✅ Therefore, the correct answer is **B. Overwrites history**.

Correct answer: **B. Enforces reviews & checks** ✅

Explanation

A **protected branch rule** in Git platforms like **GitHub/GitLab/Bitbucket** is used to **protect important branches** (like main or master).

It can enforce rules such as:

- ✅ **Pull request reviews required before merging**
- ✅ **Status checks (tests/CI) must pass**
- ✅ **Restrict who can push**
- ✅ **Prevent force pushes**

Because it **forces code reviews and checks before changes are merged**, the best answer is:

👉 **B. Enforces reviews & checks**

Why others are incorrect

- **A. Prevents deletion** – Sometimes possible but **not the main purpose**.

- **C. Blocks clone** – Cloning a repository cannot be blocked by branch protection.
- **D. Stops fetch** – Fetching is allowed; branch protection doesn't affect it.

✅ **Final Answer: B. Enforces reviews & checks**

✅ **Correct Answer: B. Pull merges automatically**

Explanation

The main difference between **git fetch** and **git pull** is how they update your local repository.

1️⃣ **git fetch**

- Downloads the **latest changes from the remote repository**
- **Does NOT merge** them into your current branch
- You must merge manually.

Example:

git fetch origin

2️⃣ **git pull**

- First **fetches the changes**
- Then **automatically merges** them into your current branch.

Example:

git pull origin main

👉 Internally:

git pull = git fetch + git merge

Why other options are wrong

- **A. Same** ❌ They are not the same.
- **C. Fetch deletes code** ❌ Fetch only downloads updates.
- **D. Pull is unsafe** ❌ Pull is safe; it just merges automatically.

✅ **Final Answer: B. Pull merges automatically**

✅ **Correct Answer: B. git init**

Explanation

The command used to **initialize a new Git repository** is:

git init

When you run this command:

- Git creates a **.git folder** inside your project.
- This folder stores **all version control information** (commits, branches, history).

Example:

```
mkdir myProject
cd myProject
git init
```

Output will be something like:

Initialized empty Git repository

Why other options are wrong

- **A. git start** ❌ Not a valid Git command
- **C. git begin** ❌ Not a Git command
- **D. git create** ❌ Not a Git command

✅ **Final Answer: B. git init**

✅ **Correct Answer: B. Use PRs**

Explanation

The **best practice for team collaboration in Git** is to use **Pull Requests (PRs)**.

A **Pull Request** allows:

- 👨‍💻 Team members to **review code before merging**
- 🐞 **Detect bugs early**
- 💬 **Discuss changes**
- 🔒 Ensure **quality control**

Typical workflow:

1. Create a **new branch**
2. Make changes
3. Push branch

4. Create a **Pull Request**
5. Team reviews and merges into main

Why other options are wrong

- **A. Push directly to main** ❌ Risky and bypasses review.
- **C. Avoid branches** ❌ Branches are essential for collaboration.
- **D. Force push** ❌ Dangerous because it can overwrite history.

✅ **Final Answer: B. Use PRs**

✅ **Correct Answer: B. Rename file**

Explanation

The code shown is a **Node.js file system (fs module)** operation.

```
const fs = require('fs');

fs.rename('old.txt', 'new.txt', (err) => {});
```

What it does:

- fs.rename() **changes the name of a file**.
- Here it **renames old.txt to new.txt**.

Breakdown

- require('fs') → imports the **File System module**
- fs.rename(oldName, newName, callback) → **renames or moves a file**

Example:

```
fs.rename('old.txt', 'new.txt', (err) => {
  if (err) throw err;
  console.log("File renamed successfully");
});
```

Why other options are wrong

- **A. Delete file** ❌ That would use fs.unlink()
- **C. Copy file** ❌ That would use fs.copyFile()
- **D. Read file** ❌ That would use fs.readFile()

✅ **Final Answer: B. Rename file**

✅ **Correct Answer: B. Improve code quality**

Explanation

Code review means **other developers check your code before it is merged into the main branch**.

It mainly helps to:

- 🐞 **Find bugs**
- 🧹 **Improve code quality**
- 📚 **Ensure coding standards**
- 🤝 **Share knowledge among team members**

Why other options are incorrect

- **A. Increase conflicts** ❌ Code review usually **reduces mistakes**, not conflicts.
- **C. Slow development** ❌ It may take time but **improves long-term development quality**.
- **D. Delete commits** ❌ Code review does not delete commits.

✅ **Final Answer: B. Improve code quality**

✅ **Correct Answer: C. Requested URL path**

Explanation

In **Node.js HTTP module**, req represents the **incoming request object**.

```
const http = require('http');

http.createServer((req, res) => {
  console.log(req.url);
  res.end();
}).listen(4000);
```

What req.url means

req.url contains the **URL path that the client requested**.

Example:

If a user visits:

http://localhost:4000/about

Then:

req.url = "/about"

If the user visits:

http://localhost:4000/contact

Then:

req.url = "/contact"

Why other options are wrong

- **A. Server port** ❌ Port is 4000 in .listen(4000)
- **B. HTTP method** ❌ That would be req.method
- **D. Hostname** ❌ That comes from headers like req.headers.host

✅ **Final Answer: C. Requested URL path**

✅ **Correct Answer: C. Resolves to absolute path**

Explanation

In **Node.js**, the path module helps work with **file and directory paths**.

Example from the question:

```
const path = require('path');

console.log(path.resolve('test', 'demo.txt'));
```

What path.resolve() does

path.resolve() **converts a sequence of paths into an absolute path** starting from the current working directory.

Example output:

C:\Users\Rahul\project\test\demo.txt

So it **resolves relative paths into a full absolute path**.

Why other options are wrong

- **A. Deletes path** ❌ No deletion happens.
- **B. Creates file** ❌ It only works with paths, not files.
- **D. Reads a path** ❌ It does not read files.

✅ **Final Answer: C. Resolves to absolute path**

✅ **Correct Answer: B. Small and focused**

Explanation

A **good Pull Request (PR)** should be:

- 🎯 **Small**
- 🎯 **Focused on one feature or fix**
- 🎯 **Easy to review**

This helps reviewers:

- Understand changes quickly
- Detect bugs easily
- Approve faster

Why other options are wrong

- **A. Huge** ❌ Large PRs are hard to review.
- **C. Without description** ❌ A good PR should always have a clear description.
- **D. Urgent** ❌ Urgency is not a quality of a good PR.

✅ **Final Answer: B. Small and focused**

✅ **Correct Answer: B. /about**

Explanation

Given code:

```
const url = new URL("http://localhost:3000/about");
console.log(url.pathname);
```

The **URL object** breaks a URL into parts:

Part	Value
protocol	http:
hostname	localhost
port	3000
pathname	/about

So when we print:

console.log(url.pathname);

The output will be:

/about

Why other options are wrong

- **A. localhost** ❌ This is url.hostname
- **C. 3000** ❌ This is url.port
- **D. http** ❌ This is part of url.protocol

✅ **Final Answer: B. /about**

✅ **Correct Answer: A. folder/file.txt**

Explanation

Code in the question:

```
const path = require('path');
console.log(path.join('folder', 'file.txt'));
```

What path.join() does

path.join() **joins multiple path segments into one path** using the correct OS separator.

So:

path.join('folder', 'file.txt')

becomes:

folder/file.txt

(On Windows it may show folder\file.txt, but logically it represents the same path.)

Why other options are wrong

- **B. folderfile.txt** ❌ No separator.
- **C. /folder/file.txt** ❌ join() does not automatically add / at the beginning.
- **D. Error** ❌ The code is valid.

✅ **Final Answer: A. folder/file.txt**

✅ **Correct Answer: B. Trunk-based development**

Explanation

Trunk-Based Development (TBD) is a strategy where developers:

- Work on **short-lived branches**
- Frequently **merge changes into the main branch (trunk)**
- Avoid **long-lived feature branches**

This helps in:

- 🚀 Faster integration
- 🤝 Less merge conflicts
- 🔄 Continuous integration

Why other options are wrong

- **A. Git Flow** ❌ Uses **long-lived branches** like develop, release, etc.
- **C. Forking** ❌ Used for open-source collaboration, not specifically for avoiding long branches.
- **D. Manual merge** ❌ Not a development strategy.

✅ **Final Answer: B. Trunk-based development**

✅ **Correct Answer: A. File is created with content "NodeJS"**

Explanation

Code given:

```
const fs = require('fs');
```

```
fs.writeFileSync('test.txt', 'NodeJS');
console.log("Done");
```

What happens here

1️⃣ fs.writeFileSync()

- Creates a file **test.txt** if it does not exist.
- Writes the text **"NodeJS"** inside the file.
- It is **synchronous**, so it completes before the next line runs.

2️⃣ After writing, the program prints:

Done

Final Result

- A file **test.txt** is created
- The file contains **NodeJS**

Why other options are wrong

- **B. File is empty** ❌ Content "NodeJS" is written.
- **C. Error occurs** ❌ No error in this code.
- **D. Nothing happens** ❌ File creation and console output happen.

✅ **Final Answer: A. File is created with content "NodeJS"**

✅ **Correct Answer: B. Appends Hello to file**

Explanation

Code given:

```
const fs = require('fs');
fs.appendFileSync('a.txt', 'Hello');
```

What appendFileSync() does

- It **adds (appends) data to the end of a file**.
- If the file **does not exist**, it **creates the file first**.

Example

If a.txt already contains:

Node

After running the code, it becomes:

NodeHello

Why other options are wrong

- **A. Deletes file** ❌ That would use fs.unlink()
- **C. Overwrites file** ❌ That is done by fs.writeFileSync()
- **D. Reads file** ❌ That is done by fs.readFileSync()

✅ **Final Answer: B. Appends Hello to file**

✅ **Correct Answer: A. /home/user**

Explanation

Code given:

```
const path = require('path');
console.log(path.dirname('/home/user/app.js'));
```

What path.dirname() does

path.dirname() returns the **directory (folder) name of a path**, removing the file name.

So:

/home/user/app.js

Breakdown:

Part	Meaning
/home/user	Directory
app.js	File

Therefore:

path.dirname('/home/user/app.js')

Output:

/home/user

Why other options are wrong

- **B. app.js** ❌ This is the file name.
- **C. /home** ❌ One level above the actual directory.
- **D. user** ❌ Only the folder name, not the full path.

✅ **Final Answer: A. /home/user**

✅ **Correct Answer: B. On POST request**

Explanation

Code given:

```
const http = require('http');

http.createServer((req, res) => {
  if (req.method === 'POST') {
    res.end('POST Request');
  }
}).listen(5000);
```

How it works

- req.method tells the **HTTP request type** (GET, POST, PUT, DELETE, etc.)
- The condition:

if (req.method === 'POST')

means the response will be sent **only when the request method is POST**.

Example

If a client sends:

POST http://localhost:5000

Server response:

POST Request

If it is a **GET request**, the condition fails and **no response is sent**.

Why other options are wrong

- **A. On GET request** ❌ Condition checks for POST.
- **C. On DELETE request** ❌ Not checked in the code.
- **D. Always** ❌ Only executes when method is POST.

✅ **Final Answer: B. On POST request** 👍

✅ **Correct Answer: B. It will print Hello in browser**

Explanation

Code given:

```
const http = require('http');

http.createServer((req, res) => {
  res.write('Hello');
  res.end();
}).listen(3000);
```

How it works

1️⃣ http.createServer() creates a **web server**.

2️⃣ When a browser requests:

http://localhost:3000

3️⃣ The server runs this code:

res.write('Hello');

This sends **"Hello"** to the browser.

4️⃣ Then:

res.end();

This **finishes the response**.

Result in Browser

The page will display:

Hello

Why other options are wrong

- **A. It will throw an error** ❌ Code is correct.
- **C. Server will stop immediately** ❌ Server keeps running on port **3000**.
- **D. Nothing will happen** ❌ Browser will show **Hello**.

✅ **Final Answer: B. It will print Hello in browser** 🌐

✅ **Correct Answer: B. Cleans path structure**

Explanation

Code:

```
const path = require('path');
console.log(path.normalize('/test//demo/../file.txt'));
```

What path.normalize() does

path.normalize() **cleans and simplifies a path** by:

- Removing **extra slashes (//)**
- Resolving **. (current folder)**
- Resolving **.. (parent folder)**

Example

Input path:

/test//demo/../file.txt

Steps:

1️⃣ // → /
2️⃣ demo/.. → removes demo

Result:

/test/file.txt

So normalize() **cleans and organizes the path structure**.

Why other options are wrong

- **A. Deletes file** ❌ It only processes the path string.
- **C. Creates path** ❌ It does not create folders/files.
- **D. Reads path** ❌ It does not read files.

✅ **Final Answer: B. Cleans path structure**

✅ **Correct Answer: C. git push**

Explanation

The command used to **upload local commits to GitHub (remote repository)** is:

git push

Workflow in Git

1️⃣ Add files

git add .

2️⃣ Commit changes

git commit -m "message"

3️⃣ Upload commits to GitHub

git push origin main

Why other options are wrong

- **A. git pull** ❌ Downloads changes from GitHub.
- **B. git fetch** ❌ Retrieves updates but does not merge them.

- **D. gitupload** ❌ Not a valid Git command.

✅ **Final Answer: C. git push** 🚀

✅ **Correct Answer: A. main, develop**

Explanation

In the **Git Flow branching model**, the two main permanent branches are:

1️⃣ **main (or master)**

- Contains **production-ready code**
- Stable version of the project

2️⃣ **develop**

- Used for **active development**
- Features are merged here before going to main

Other temporary branches in Git Flow:

- **feature branches**
- **release branches**
- **hotfix branches**

Why other options are wrong

- **B. test, prod** ❌ Not Git Flow branch names.
- **C. local, remote** ❌ These describe repository types, not branches.

- **D. ui, api** ❌ These are project modules, not Git branches.

✅ **Final Answer: A. main, develop**

✅ **Correct Answer: C. Synchronous blocking**

Explanation

Code:

```
const fs = require('fs');
fs.readFileSync('data.txt');
```

What readFileSync() does

- The word **Sync** means **synchronous**.
- The program **waits until the file is completely read** before moving to the next line.

So during this operation:

- ⏳ The program **blocks execution**
- Nothing else runs until the file reading is finished.

Example Flow

1️⃣ Program starts
2️⃣ readFileSync() reads data.txt
3️⃣ Program **waits (blocks)** until reading is done
4️⃣ Then the next code executes

Why other options are wrong

- **A. Asynchronous** ❌ That would be fs.readFile()

- **B. Event driven** ❌ Not related to this function
- **D. Stream** ❌ Streams use createReadStream()

✅ **Final Answer: C. Synchronous blocking**

✅ **Correct Answer: B. Distributed**

Explanation

Git is a **Distributed Version Control System (DVCS)**.

What Distributed means

- Every developer has a **complete copy of the repository**.
- The full **history of commits** exists on each local machine.
- Work can be done **offline** without connecting to a central server.

Example Workflow

```
git clone repo
git add .
git commit -m "update"
git push
```

Developers work **locally** and later **sync with remote repositories** like GitHub.

Why other options are wrong

- **A. Centralized** ❌ Used by systems like **SVN**.
- **C. Local only** ❌ Git can work with **remote repositories**.
- **D. Cloud-based** ❌ Git itself is not cloud-based; platforms like **GitHub/GitLab** host repositories in the cloud.

✅ **Final Answer: B. Distributed**

✅ **Correct Answer: B. Project storage**

Explanation

A **repository (repo)** in Git is a **place where your project files and their version history are stored**.

It contains:

- 📁 Project files
- 📝 Commit history
- 🌿 Branch information
- 🔁 Version changes over time

Example repositories:

- **Local repository** (on your computer)
- **Remote repository** (on platforms like GitHub, GitLab, Bitbucket)

Why other options are wrong

- **A. Code editor** ❌ Editors are tools like VS Code.
- **C. Server** ❌ A repo can be on a server, but it is not the server itself.
- **D. Branch** ❌ Branch is a part inside a repository.

✅ **Final Answer: B. Project storage** 📦

✅ **Correct Answer: A. git switch**

Explanation

The command used to **switch to an existing branch** is:

git switch branch_name

Example:

git switch feature-login

This moves your working directory from the current branch to **feature-login**.

Note

Traditionally, developers also used:

git checkout branch_name

But newer Git versions introduced **git switch** to make branch switching clearer.

Why other options are wrong

- **B. git move** ❌ Not a valid Git command
- **C. git change** ❌ Not a Git command
- **D. git merge** ❌ Used to combine branches, not switch

✅ **Final Answer: A. git switch** 🌿

✅ **Correct Answer: A. Fast-forward**

Explanation

A **Fast-forward merge** keeps the **Git history clean and linear**.

How it works

If the target branch has **no new commits**, Git simply **moves the branch pointer forward** instead of creating a merge commit.

Example:

Before merge:

```
main:    A — B
          \
feature:     C — D
```

After **fast-forward merge**:

```
main: A — B — C — D
```

✓ No extra **merge commit**
✓ Clean and simple history

Why other options are wrong

- **B. Recursive** ❌ Default merge strategy but creates merge commits.
- **C. Octopus** ❌ Used to merge **multiple branches at once**.
- **D. Ours** ❌ Keeps current branch changes and ignores others.

✅ **Final Answer: A. Fast-forward**

✅ **Correct Answer: B. Pull latest changes**

Explanation

Before pushing your code, you should **pull the latest changes from the remote repository**.

Command:

```
git pull origin main
```

Why this is important

- 🔄 Syncs your local repository with the remote repository
- ⚠️ Prevents **merge conflicts**
- 👥 Ensures you are working on the **latest code from teammates**

Typical workflow

```
git pull
git add .
git commit -m "your changes"
git push
```

Why other options are wrong

- **A. Delete branch** ❌ Not required before pushing.

- **C. Rebase randomly** ❌ Rebasing should be done carefully, not randomly.
- **D. Reset hard** ❌ Dangerous; it deletes local changes.

✅ **Final Answer: B. Pull latest changes** 🔄

✅ **Correct Answer: B. Commit history**

Explanation

The command:

git log

displays the **history of commits in the repository**.

It shows details like:

- 🆔 **Commit hash**
- 👤 **Author name**
- 📅 **Date**
- 📝 **Commit message**

Example Output

```
commit a3f5b9c
Author: Rahul
Date: Mon Mar 6

Added login feature
```

Why other options are wrong

- **A. Current branch** ❌ That is shown by git branch.
- **C. File changes** ❌ That is shown by git diff.
- **D. Staged files** ❌ That is shown by git status.

✅ **Final Answer: B. Commit history** 📜

✅ **Correct Answer: B. Akki**

Explanation

Code:

```
const url = require('url');

const myUrl = new URL('http://localhost:3000/test?name=Akki');

console.log(myUrl.searchParams.get('name'));
```

Breakdown of the URL

http://localhost:3000/test?name=Akki

Part	Value
Protocol	http
Host	localhost
Port	3000
Path	/test
Query parameter	name=Akki

What searchParams.get() does

myUrl.searchParams.get('name')

- It **retrieves the value of the query parameter name**.
- The value is **"Akki"**.

Output

Akki

Why other options are wrong

- **A. test** ❌ That is the pathname.
- **C. localhost** ❌ That is the hostname.
- **D. undefined** ❌ The parameter name exists.

✅ **Final Answer: B. Akki**

✅ **Correct Answer: B. git add**

Explanation

The command used to **stage files in Git** is:

git add filename

or to stage all files:

git add .

Git Workflow

1️⃣ **Working Directory** → where you edit files
2️⃣ **Staging Area** → files prepared for commit (git add)
3️⃣ **Repository** → saved commits (git commit)

Example:

```
git add index.js
git commit -m "Added new feature"
git push
```

Why other options are wrong

- **A. git stage** ❌ Not a valid Git command
- **C. git commit** ❌ Saves staged changes to repository
- **D. git push** ❌ Uploads commits to remote repository

✅ **Final Answer: B. git add** 📂

✅ **Correct Answer: B. Apply specific commit**

Explanation

git cherry-pick is used to **apply a specific commit from one branch to another branch**.

Example:

```
git cherry-pick <commit-id>
```

This command **copies the changes from that particular commit** and applies them to the current branch.

Example Scenario

Branch structure:

```
main:    A --- B --- C
feature: A --- B --- D
```

If you want commit **D** in main without merging the entire feature branch:

```
git checkout main
git cherry-pick D
```

Result:

```
main: A --- B --- C --- D
```

Why other options are wrong

- **A. Merge all commits** ❌ That is done with git merge.

- **C. Delete history** ❌ Cherry-pick does not delete history.
- **D. Rebase branch** ❌ That is done with git rebase.

✅ **Final Answer: B. Apply specific commit**

✅ **Correct Answer: C. If file exists**

Explanation

Code:

```
const fs = require('fs');

fs.exists('a.txt', (exists) => {
  console.log(exists);
});
```

What fs.exists() does

- It **checks whether a file or path exists**.
- The callback receives a **boolean value**:
  - true → file exists

- false → file does not exist

Example

If a.txt exists:

true

If a.txt does not exist:

false

Why other options are wrong

- **A. File size** ❌ That uses fs.stat()
- **B. File content** ❌ That uses fs.readFile()
- **D. File type** ❌ Not checked by exists()

✅ **Final Answer: C. If file exists**

✅ **Correct Answer: B. main**

Explanation

When you initialize a Git repository using:

git init

Git creates a **default branch called main** (in modern Git versions).

This branch is the **primary branch** where the stable code of the project is stored.

Example

```
git init
git branch
```

Output:

* main

Note

Earlier Git versions used **master** as the default branch, but now most platforms (like GitHub) use **main**.

Why other options are wrong

- **A. dev** ❌ Created manually for development.
- **C. feature** ❌ Feature branches are created manually.
- **D. test** ❌ Not a default Git branch.

✅ **Final Answer: B. main**

✅ **Correct Answer: B. Commit history**

Explanation

git rebase is used to **rewrite the commit history**.

It works by:

- Taking commits from one branch
- Reapplying them on top of another branch

Example:

Before rebase:

```
main:   A --- B --- C
           \
feature:      D --- E
```

Command:

git rebase main

After rebase:

```
main:   A --- B --- C
               \
feature:          D' --- E'
```

Notice:

- Commits **D and E are recreated (rewritten)**.
- Their **commit IDs change**, which means **history is rewritten**.

Why other options are wrong

- **A. Working tree** ❌ Rebase does not rewrite the working directory structure.
- **C. Remote repo** ❌ Rebase works on local commits.
- **D. Tags** ❌ Tags are not rewritten by rebase.

✅ **Final Answer: B. Commit history** 📜

✅ **Correct Answer: B. Callback function**

Explanation

Code given:

```
const http = require('http');
const server = http.createServer();
server.listen(8000);
```

Here, the server is created but **no function is provided to handle requests**.

Normally, createServer() needs a **callback function** that handles req (request) and res (response).

Correct version

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.write("Hello");
  res.end();
});

server.listen(8000);
```

What the callback does

The callback:

```
(req, res) => { ... }
```

- Handles **incoming requests**
- Sends **responses back to the client**

Why other options are wrong

- **A. Port number** ❌ Already provided (8000).
- **C. Module import** ❌ http module is already imported.
- **D. File path** ❌ Not needed for handling HTTP requests.

✅ **Final Answer: B. Callback function**

✅ **Correct Answer: A. true**

Explanation

Code:

```
const path = require('path');
console.log(path.isAbsolute('/test/file'));
```

What path.isAbsolute() does

path.isAbsolute() checks whether a given path is an **absolute path**.

- Returns **true** → if the path is absolute
- Returns **false** → if the path is relative

Example

```
path.isAbsolute('/test/file')   // true
path.isAbsolute('test/file')    // false
```

Since:

/test/file

starts from the **root directory (/)**, it is an **absolute path**.

Why other options are wrong

- **B. false** ❌ Path starts with /, so it is absolute.
- **C. undefined** ❌ The function returns a boolean.

- **D. error** ❌ Code syntax is correct.

✅ **Final Answer: A. true**

✅ **Correct Answer: C. Microsoft**

Explanation

GitHub was originally created by **Tom Preston-Werner, Chris Wanstrath, and PJ Hyett**.

In **2018**, **Microsoft acquired GitHub** for about **$7.5 billion**.

So now **GitHub is owned by Microsoft**.

Why other options are wrong

- **A. Google** ❌ Google owns platforms like **YouTube**.
- **B. Amazon** ❌ Amazon owns **AWS**.
- **D. Meta** ❌ Meta owns **Facebook, Instagram, WhatsApp**.

✅ **Final Answer: C. Microsoft** 💻

✅ **Correct Answer: A. GitHub**

Explanation

GitHub is a platform used for **hosting Git repositories online**.
It allows developers to:

- 📂 Store code repositories
- 🤝 Collaborate with teams
- 🔀 Create **pull requests**
- 🐞 Track issues
- 🔄 Manage version control

Why other options are wrong

- **B. Docker** ❌ Used for **containerization**, not Git hosting.
- **C. Jenkins** ❌ Used for **CI/CD automation**.
- **D. Postman** ❌ Used for **API testing**.

✅ **Final Answer: A. GitHub** 🌐

✅ **Correct Answer: B. Developer**

Explanation

A **merge conflict** happens when Git cannot automatically merge changes because the **same part of a file was modified differently in two branches**.

Git detects the conflict, but **it cannot decide which change is correct**.

So the **developer must manually resolve the conflict**.

Example conflict

```
<<<<<<< HEAD
console.log("Version A");
=======
console.log("Version B");
>>>>>>> feature
```

The **developer edits the file** to choose the correct version and then commits the fix.

Why other options are wrong

- **A. GitHub** ❌ GitHub only shows the conflict; it doesn't decide.
- **C. Server** ❌ Servers do not resolve code conflicts.
- **D. Git** ❌ Git detects conflicts but cannot resolve them automatically.

✅ **Final Answer: B. Developer** 👨‍💻

✅ **Correct Answer: B. 8080**

Explanation

Code:

```
const url = new URL("https://example.com:8080/path");
console.log(url.port);
```

URL Breakdown

https://example.com:8080/path

Part	Value
Protocol	https
Hostname	example.com
Port	8080
Pathname	/path

What url.port returns

url.port returns the **port number used in the URL**.

So:

```
url.port
```

Output:

8080

Why other options are wrong

- **A. https** ❌ That is url.protocol
- **C. path** ❌ That is url.pathname
- **D. example.com** ❌ That is url.hostname

✅ **Final Answer: B. 8080** 🔢

✅ **Correct Answer: B. Single commit**

Explanation

A **squash merge** combines **multiple commits from a branch into one single commit** before merging it into the target branch.

Example

Feature branch commits:

A --- B --- C --- D

After **squash merge** into main:

A --- B --- E

Here:

- C and D are **combined into one commit E**.

Why squash merge is used

- 🧹 Keeps **Git history clean**

- 📜 Avoids many small commits
- 🎯 Creates **one meaningful commit**

Why other options are wrong

- **A. Multiple commits** ❌ Squash reduces them to one.
- **C. Conflict** ❌ Not the purpose of squash merge.
- **D. Rebases** ❌ Squash is different from rebase.

✅ **Final Answer: B. Single commit**

✅ **Correct Answer: A. Create directory**

Explanation

Code:

```
const fs = require('fs');

fs.mkdir('testDir', (err) => {});
```

What fs.mkdir() does

fs.mkdir() is used to **create a new directory (folder)**.

In this example:

- It creates a folder named **testDir** in the current directory.

Example result

Before running code:

```
project/
```

After running code:

```
project/
  testDir/
```

Why other options are wrong

- **B. Delete directory** ❌ That would use fs.rmdir()
- **C. Read directory** ❌ That would use fs.readdir()
- **D. Rename directory** ❌ That would use fs.rename()

✅ **Final Answer: A. Create directory** 📁

✅ **Correct Answer: B. git branch**

Explanation

The command used to **list all branches in a Git repository** is:

```
git branch
```

Example Output

```
* main
  feature-login
  dev
```

- * indicates the **current active branch**.

Other useful branch commands

```
git branch -a     # show all local + remote branches
git branch -r     # show remote branches
```

Why other options are wrong

- **A. git show** ❌ Shows details of commits/tags.
- **C. git list** ❌ Not a valid Git command.
- **D. gittree** ❌ Not a Git command.

✅ **Final Answer: B. git branch** 🌿

✅ **Correct Answer: C. It reads file asynchronously**

Explanation

Code:

```
const fs = require('fs');

fs.readFile('abc.txt', 'utf8', (err, data) => {
  console.log(data);
});
```

What fs.readFile() does

- fs.readFile() is an **asynchronous function**.
- It reads the file **without blocking the event loop**.
- When the file is read, the **callback function executes**.

Flow

1️⃣ Program calls readFile()
2️⃣ Node.js continues executing other tasks
3️⃣ When file reading finishes → callback runs
4️⃣ console.log(data) prints the file content

Why other options are wrong

- **A. Reads file synchronously** ❌ That would be fs.readFileSync().
- **B. Blocks event loop** ❌ Asynchronous functions **do not block**.
- **D. Deletes file** ❌ File deletion uses fs.unlink().

✅ **Final Answer: C. It reads file asynchronously**

✅ **Correct Answer: B. Causes history rewrite issues**

Explanation

git rebase **rewrites commit history** by creating new commits with new commit IDs.

If you rebase a **shared branch** (a branch that other developers are using), it can cause problems because:

- 🔄 Commit history changes
- ⚠️ Other developers' commits may not match the rewritten history
- 🚫 Leads to **conflicts and push problems**

Example:

Before rebase:

A --- B --- C

After rebase:

A --- B --- C'

Commit **C becomes C' (new commit ID)**.

If others already pulled **C**, their history becomes inconsistent.

Why other options are wrong

- **A. Slow** ❌ Rebase is not avoided because of speed.
- **C. Deletes files** ❌ Rebase does not delete files.
- **D. Cannot push** ❌ You can push, but often need **force push**.

✅ **Final Answer: B. Causes history rewrite issues**